

SignNet

Gebärdensprachübersetzung mit Computer Vision & Deep Learning

ABSCHLUSSARBEIT CAS MACHINE LEARNING FOR SOFTWARE ENGINEERS (ML4SE)

ANDREI CHIRILĂ
ROMAN SCHLÄPFER
Januar 2026

Unter der Aufsicht von:

Martin STYPINSKI
Kursleiter CAS ML4SE



CAS
Machine Learning for
Software Engineers

Abstract

Diese Arbeit untersucht, inwieweit Transformer-Modelle zur automatischen Erkennung von Gebärdensprache auf Basis von mit MediaPipe extrahierten Landmark-Sequenzen geeignet sind. Ziel war es, einen belastbaren Klassifikator zu entwickeln, der sowohl mit Klassenungleichgewichten als auch mit visuellen Ähnlichkeiten zwischen Zeichen zuverlässig umgehen kann. Als Datengrundlage dient der RWTH-PHOENIX Weather 2014 Datensatz [7], ergänzt durch abgeleitete Merkmale wie Geschwindigkeits- und Knochenvektoren, die zusätzliche dynamische und strukturelle Informationen bereitstellen. Methodisch verbinden wir gezieltes Feature-Engineering mit einer angepassten Transformer-Architektur und einer hierarchisch organisierten Inferenzstrategie. Die durchgeführten Experimente zeigen gegenüber Standardarchitekturen eine klare Verbesserung der Klassifikationsgenauigkeit und eine erhöhte Robustheit insbesondere bei seltenen Klassen. Abschliessend analysieren wir die wichtigsten Fehlerquellen und diskutieren konkrete Ansätze zur Verbesserung. Die gewonnenen Erkenntnisse liefern Anhaltspunkte für weiterführende Forschung und stärken die Anwendbarkeit der Methode in praktischen Systemen zur Gebärdensprachübersetzung.

Inhaltsverzeichnis

1	Einleitung	2
1.1	Thematik und Relevanz	2
1.2	Problemstellung	2
1.3	Zielsetzung	2
2	Stand der Technik	3
2.1	Traditionelle Methoden	3
2.2	Deep-Learning-Ansätze	3
2.2.1	CNN-RNN-Architekturen	3
2.2.2	3D-CNNs	3
2.2.3	Transformer-Architekturen	3
2.3	Skelettbasierte Ansätze	3
2.3.1	Pose-Estimation	4
2.3.2	Graph Convolutional Networks	4
2.4	Datensätze	4
2.4.1	WLASL	4
2.4.2	PHOENIX	4
2.5	Herausforderung: Klassenungleichheit	4
2.5.1	Focal Loss	4
2.5.2	Class-Balanced Loss	4
2.5.3	Balanced Softmax	5
2.6	Zusammenfassung	5
3	Hintergrund	6
3.1	Gebärdensprache als linguistisches System	6
3.1.1	Phonologie der Gebärdensprache	6
3.1.2	Glossennotation	6
3.1.3	Isolierte vs. kontinuierliche Gebärdenerkennung	6
3.2	Transformer-Architektur	7
3.2.1	Self-Attention	7
3.2.2	Positional Encoding	7
3.2.3	Vorteile gegenüber rekurrenten Architekturen	7
3.3	Herausforderungen des Datensatzes	8
3.3.1	Long-Tail-Verteilung	8
3.3.2	Gini-Koeffizient	8
3.3.3	Visuelle Konfusion	8
4	Methodik	9
4.1	Datengrundlage und Klassenwahl	9
4.1.1	Filterkriterien und Stichprobenumfang	10
4.1.2	Merkmalsextraktion mit MediaPipe	10
4.2	Iterativer Forschungsansatz und Versuchsaufbau	10
4.2.1	Phase 1: Statische Baseline (CNN) – Fingeralphabet	10
4.2.2	Phase 2: Wechsel zum PHOENIX-Datensatz (Satzebene)	10
4.2.3	Phase 3: Minimalistisches Satz-Modell (Debugging)	10
4.2.4	Phase 4: Übergang zu isolierten Wörtern (Top 10)	11
4.2.5	Phase 5: Skalierung auf Top 50 Wörter	11
4.2.6	Phase 6: Skalierung auf 146 Klassen und Wechsel zum Transformer	11

4.2.7	Phase 7: Ein Exkurs in skelettbasierte Graphen	11
4.2.8	Phase 8: Rückkehr zur optimierten Transformer-Architektur (v2)	11
4.2.9	Phase 9: Finales optimiertes Modell	11
4.3	Vorverarbeitung und Datenreinigung	11
4.3.1	Umgang mit Fehlwerten und Verdeckungen (Occlusion)	12
4.4	Datenaugmentation	12
4.4.1	Temporale Augmentation (TemporalAugmentation)	12
4.4.2	Landmark-Occlusion-Augmentation	12
4.4.3	Mirror Augmentation	12
4.4.4	Weighted Sampling und Oversampling	12
4.5	Feature Engineering: Kinematik und Geometrie	12
4.5.1	Kinematische Ableitungen	13
4.5.2	Skelett-Geometrie und Bone-Vektoren	13
4.5.3	Distanzmerkmale und Normalisierung	13
4.6	Modellarchitektur: Transformer-Encoder	13
4.6.1	Spezifikation der Transformer-Architektur	13
5	Resultate	15
5.1	Vergleich der Entwicklungsphasen	15
5.2	Datensatz-Statistik und Split-Verhältnis	15
5.3	Analyse des Trainingsverlaufs	15
5.4	Quantitative Ergebnisse	16
5.5	Konfidenz-Analyse und Kalibrierung	16
5.6	Detaillierte Fehleranalyse	17
5.6.1	Evaluation nach Klassenhäufigkeit (Buckets)	18
6	Diskussion	19
6.1	Interpretation der Klassifikationsleistung	19
6.1.1	Robustheit gegenüber Klassenungleichgewicht (Bucket-Analyse)	19
6.2	Evaluation der Modellstabilität und Generalisierungsfähigkeit	19
6.2.1	Detaillierte Loss-Analyse	19
6.2.2	Interpretation des Accuracy-Gaps und Early Stopping	19
6.2.3	Fazit zur Generalisierung	20
6.3	Analyse linguistischer Konfusionscluster	20
6.4	Modellstabilität und Konfidenz-Evaluation	20
6.5	Methodische Reflexion: Das Validierungs-Dilemma	20
6.5.1	Statistische Relevanz vs. Unabhängigkeit	20
6.5.2	Risiko des Hyperparameter-Overfittings	20
6.6	Architektonische Einflüsse und Limitationen	21
7	Fazit und Ausblick	22
7.1	Zusammenfassung der Ergebnisse	22
7.2	Technische Würdigung	22
7.3	Ausblick	22
7.4	Persönliches Fazit Andrei	22
7.4.1	Was habe ich gut gemacht?	23
7.4.2	Was würde ich anders machen?	23
7.4.3	Was habe ich gelernt?	23
7.5	Persönliches Fazit Roman	24
7.5.1	Was habe ich gut gemacht?	24
7.5.2	Was würde ich anders machen?	24
7.5.3	Was habe ich gelernt?	24
	Literaturverzeichnis	25

Einleitung

1.1 Thematik und Relevanz

Gebärdensprachen sind eigenständige, vollwertige Sprachen mit eigener Grammatik und Syntax. Weltweit nutzen Millionen gehörloser und schwerhöriger Menschen Gebärden als primäres Kommunikationsmittel. Gleichzeitig besteht in vielen Alltagssituationen eine deutliche Kommunikationsbarriere zwischen Gehörlosen und Hörenden, weil nur ein kleiner Teil der hörenden Bevölkerung Gebärdensprache beherrscht. Trotz der grossen Nutzerzahl fehlen daher vielfach barrierefreie Lösungen – sei es in öffentlichen Einrichtungen, im Gesundheitswesen oder in Bildungseinrichtungen.

Automatische Systeme zur Erkennung von Gebärdensprach-Glossen (Automatic Sign Language Recognition, ASLR) könnten diese Lücke schliessen. Fortschritte in Computer Vision und Deep Learning ermöglichen heute, visuelle Informationen aus Videosequenzen zuverlässig zu extrahieren und zu klassifizieren, sodass technologische Übersetzungshilfen zunehmend realisierbar werden.

1.2 Problemstellung

Die Herausforderung liegt in der Multimodalität der Gebärdensprache: Informationen werden gleichzeitig über Handbewegungen, Handform, Körperhaltung und Mimik vermittelt. Eine einzelne Gebärde entsteht aus der kombinatorischen Wirkung von bis zu fünf Parametern (Handform, Handstellung, Ausführungsart, Bewegung und non-manuelle Komponenten).

Aus technischer Sicht führen diese Eigenschaften zu hochdimensionalen, zeitlich variablen Datenströmen. Relevante Merkmale müssen räumlich und temporal modelliert werden; ein einzelnes Videoframe kann bereits Hunderte von Landmark-Koordinaten (Hände, Körper, Gesicht) enthalten, die über unterschiedlich lange Sequenzen hinweg analysiert werden müssen.

Hinzu kommt die starke Klassenungleichheit in verfügbaren Datensätzen: Viele Begriffe treten sehr häufig auf, während die Mehrheit der Gebärden selten ist (*Long-Tail-Verteilung*). Diese Imbalance erschwert das Training tiefer neuronaler Netze, da Modelle dazu neigen, seltene Klassen zu vernachlässigen. Zusätzlich führen phonologische Ähnlichkeiten zwischen Gebärden – etwa bei Richtungsangaben oder semantisch verwandten Begriffen – zu systematischen Verwechslungen, weil sich die Gebärden oft nur in kleinen Bewegungsdetails unterscheiden.

1.3 Zielsetzung

Ziel dieser Arbeit ist die Entwicklung, Implementierung und Evaluation des *SignNet*-Systems zur Erkennung von Gebärdensprach-Glossen.

Konkret werden folgende Ziele verfolgt:

- **Robuste Feature-Extraction-Pipeline:** Extraktion und Aufbereitung von Körper-, Hand- und Gesichtslandmarks mittels MediaPipe für nachfolgendes maschinelles Lernen.
- **Transformer-basierter Klassifikator:** Einsatz von Attention-Mechanismen zur Modellierung temporaler Abhängigkeiten in Gebärdensequenzen.
- **Strategien gegen Klassenungleichheit:** Evaluation von Methoden wie Focal Loss, Balanced Softmax und Oversampling.
- **Evaluation auf dem Phoenix-Datensatz:** Quantitative Bewertung der Systemleistung auf einem etablierten Benchmark für die Deutsche Gebärdensprache.

Die Arbeit soll aufzeigen, inwiefern moderne Deep-Learning-Ansätze praktisch einsetzbar sind und welche architektonischen Entscheidungen die Klassifikationsleistung massgeblich beeinflussen.

Stand der Technik

Die automatische Gebärdenspracherkennung (Isolated Sign Language Recognition, ISLR) hat sich in den letzten Jahrzehnten deutlich weiterentwickelt – von handzentrierten, statistischen Verfahren hin zu komplexen Deep-Learning-Architekturen [6].

2.1 Traditionelle Methoden

Vor dem Durchbruch des Deep Learning stützten sich Erkennungssysteme vorwiegend auf handgefertigte Merkmale, etwa relative Handpositionen oder Abstände zwischen Händen und bestimmten Körperpunkten [16]. Zur Klassifikation wurden klassische Methoden wie Hidden Markov Models (HMMs), Support Vector Machines (SVMs) und k-Nearest Neighbors (kNNs) eingesetzt [13, 16]. Solche Ansätze litten jedoch unter eingeschränkter Generalisierbarkeit und limitiertem Skalierungspotenzial [1].

2.2 Deep-Learning-Ansätze

Mit der Verbreitung neuronaler Netze verlagerte sich der Fokus auf die automatische Merkmalsextraktion direkt aus Videodaten, was die Leistungsfähigkeit vieler Systeme deutlich steigerte.

2.2.1 CNN-RNN-Architekturen

Frühe Deep-Learning-Modelle kombinierten Convolutional Neural Networks (CNNs) zur räumlichen Merkmalsextraktion pro Frame mit rekurrenten Netzwerken wie LSTMs oder bidirektionalen LSTMs (Bi-LSTMs) zur Modellierung zeitlicher Dynamik [11]. Diese Kombination ermöglichte eine robuste Erfassung lokaler Bildmerkmale und ihrer zeitlichen Entwicklung.

2.2.2 3D-CNNs

3D-CNNs adressieren kurzzeitige zeitliche Abhängigkeiten direkt in der Faltungsebene. Das Inflated 3D-CNN (I3D) von Carreira und Zisserman [4] zählt zu den am weitesten verbreiteten Architekturen für Action- und Gebärdenerkennung. Allerdings zeigen solche Modelle oft Schwächen bei der Erfassung langfristiger Abhängigkeiten über längere Sequenzen.

2.2.3 Transformer-Architekturen

Jüngst gewinnen Transformer-Modelle an Bedeutung, da ihr Self-Attention-Mechanismus Abhängigkeiten über beliebig lange Sequenzen modellieren kann [15]. Für die Gebärdenerkennung bedeutet das: Alle Frames einer Sequenz können simultan berücksichtigt werden, ohne die Limitierungen rekurrenter Modelle.

2.3 Skelettbasierte Ansätze

Statt rohe RGB-Frames zu verarbeiten, setzen skelettbasierte Methoden auf extrahierte Keypoints von Körper, Händen und Gesicht. Dadurch sinkt die Datendimensionalität, und das Lernen fokussiert sich stärker auf relevante Bewegungsmuster.

2.3.1 Pose-Estimation

Die Gewinnung von Skelettdaten erfolgt meist mit spezialisierten Pose-Estimation-Systemen. OpenPose [3] war eines der ersten Echtzeit-Systeme für Multi-Person-Pose-Estimation. MediaPipe [10] stellt eine effiziente Pipeline zur Verfügung, die Körper-, Hand- und Gesichtslandmarks extrahiert — insgesamt bis zu 553 Keypoints pro Frame. Die Nutzung von Landmark-Koordinaten statt Rohbildern bietet Vorteile wie Robustheit gegenüber Hintergrundvariationen, geringeren Speicherbedarf und schnellere Inferenz.

2.3.2 Graph Convolutional Networks

Das Spatial-Temporal Graph Convolutional Network (ST-GCN) von Yan et al. [17] war ein wichtiger Schritt zur Modellierung raumzeitlicher Dynamik in Skelettdaten. Hierbei wird das Skelett als Graph abgebildet, Gelenke sind Knoten und anatomische Verbindungen Kanten. Frühere GCN-Modelle beschränkten sich jedoch oft auf natürliche, biologische Verbindungen und vernachlässigten damit Interaktionen zwischen nicht direkt verbundenen Punkten (etwa zwischen beiden Händen).

2.4 Datensätze

Gut annotierte Datensätze sind eine Voraussetzung für das Training leistungsfähiger Modelle.

2.4.1 WLASL

Der Word-Level American Sign Language (WLASL)-Datensatz [8] umfasst über 2000 ASL-Gestiken und zählt zu den grössten öffentlich verfügbaren Sammlungen für isolierte Gebärdenerkennung. Er zeichnet sich durch hohe Variation in Aufnahmequalität und Hintergrund aus, da die Daten aus diversen Online-Quellen stammen.

2.4.2 PHOENIX

Der RWTH-PHOENIX-Weather-2014 Datensatz [7, 2] besteht aus Aufnahmen professioneller Gebärdensprachdolmetscher in Wettersendungen. Er enthält ca. 1200 kontinuierliche Gebärdensequenzen mit Gloss-Annotationen und deutschen Übersetzungen. Die konsistenten Aufnahmebedingungen sind ein Vorteil, jedoch ist die Domäne auf Wettervokabular beschränkt.

2.5 Herausforderung: Klassenungleichheit

Ein zentrales Problem in der Gebärdenerkennung ist die ungleiche Verteilung von Trainingsbeispielen über Klassen, das Long-Tail-Problem. Wenige häufige Gebärden dominieren die Datensätze, während die Mehrheit der Klassen selten vertreten ist.

2.5.1 Focal Loss

Lin et al. [9] führten Focal Loss ein, um das Ungleichgewicht zwischen einfachen und schwierigen Beispielen zu mildern. Durch den Faktor $(1 - p_t)^\gamma$ wird der Beitrag gut klassifizierter Beispiele reduziert, sodass das Training schwieriger Fälle stärker gewichtet.

2.5.2 Class-Balanced Loss

Cui et al. [5] schlugen eine Gewichtung auf Basis der *effektiven Anzahl* von Beispielen vor. Diese Methode berücksichtigt, dass zusätzliche Datenpunkte einer bereits gut vertretenen Klasse weniger Informationsgewinn bringen als für seltene Klassen.

2.5.3 Balanced Softmax

Ren et al. [12] erweiterten die Standard-Softmax-Funktion um klassenspezifische Prior-Terme, die die Trainingsklassenverteilung ausgleichen. Dadurch lassen sich seltene Klassen fairer bewerten, ohne die Verlustfunktion grundlegend zu verändern.

2.6 Zusammenfassung

Der Stand der Technik in der isolierten Gebärdenerkennung kombiniert heute skelettbasierte Merkmalsextraktion (z. B. mit MediaPipe), moderne Modellarchitekturen wie Transformer zur temporalen Modellierung und spezialisierte Verlustfunktionen zur Behandlung unbalancierter Datensätze.

Dieses Kapitel vermittelt die theoretischen Grundlagen, die zum Verständnis der vorliegenden Arbeit erforderlich sind. Zunächst wird Gebärdensprache als eigenständiges linguistisches System beschrieben; anschliessend folgen die technischen Grundlagen der verwendeten Deep-Learning-Architektur sowie Aspekte der Merkmalsextraktion.

3.1 Gebärdensprache als linguistisches System

Gebärdensprachen sind vollwertige, natürliche Sprachen mit eigener Grammatik, Syntax und Morphologie. Entgegen verbreiteten Missverständnissen handelt es sich nicht um eine visuelle Umschrift gesprochener Sprache oder um Pantomime, sondern um autonome sprachliche Systeme [14].

3.1.1 Phonologie der Gebärdensprache

William Stokoe legte 1960 den Grundstein für die linguistische Analyse von Gebärdensprachen, indem er zeigte, dass Gebärden, ähnlich wie gesprochene Wörter, aus kleineren, bedeutungsunterscheidenden Einheiten bestehen. Diese Einheiten werden als *Cheremes* (analog zu Phonemen) bezeichnet; heute spricht man häufiger von *Parametern*.

Eine Gebärde besteht simultan aus fünf Parametern:

- **Handform:** Die Fingerkonfiguration (z. B. flache Hand, Faust, ausgestreckter Zeigefinger). In der Deutschen Gebärdensprache (DGS) existieren etwa 30 distinktive Handformen.
- **Handstellung (Orientierung):** Die Richtung von Handfläche und Fingern relativ zum Körper (z. B. Handfläche nach oben, nach unten, zum Körper).
- **Ausführungsort:** Die Position im Gebärdenraum, an der die Gebärde stattfindet (z. B. vor der Brust, am Kopf, im neutralen Raum).
- **Bewegung:** Die Art und Richtung der Handbewegung (z. B. linear, kreisförmig, wiederholend).
- **Non-manuelle Komponenten:** Mimik, Mundbild sowie Kopf- und Körperhaltung, die grammatische oder lexikalische Informationen vermitteln.

Die gleichzeitige Kombination dieser Parameter unterscheidet Gebärdensprachen grundlegend von sequentiell aufgebauten Lautsprachen und stellt besondere Anforderungen an automatische Erkennungssysteme.

3.1.2 Glossennotation

In der Gebärdensprachforschung werden Gebärden häufig durch *Glossen* dargestellt. Das sind standardisierte Bezeichnungen in Grossbuchstaben, die sich an der Bedeutung orientieren. Die Glosse HAUS bezeichnet beispielsweise das Konzept „Haus“, unabhängig von variierenden Artikulationsformen. Diese Notation ist zentral für die Annotation von Videodaten und bildet die Grundlage für überwacht maschinelles Lernen.

3.1.3 Isolierte vs. kontinuierliche Gebärdenerkennung

Man unterscheidet zwei wesentliche Aufgaben in der automatischen Gebärdenerkennung:

- **Isolierte Gebärdenerkennung (Isolated Sign Language Recognition, ISLR):** Die Klassifikation einzelner, bereits segmentierter Gebärden; Anfang und Ende der Gebärde sind bekannt.
- **Kontinuierliche Gebärdenerkennung (Continuous Sign Language Recognition, CSLR):** Die Erkennung von Gebärdensequenzen ohne vorgegebene Segmentierung, vergleichbar mit der automatischen Spracherkennung.

Die vorliegende Arbeit konzentriert sich auf die isolierte Gebärdenerkennung, bei der vorgeschnittene Videosequenzen einzelnen Glossen zugeordnet werden.

3.2 Transformer-Architektur

Die Transformer-Architektur [15] gilt heute als führendes Modell für die Verarbeitung sequentieller Daten. Statt wie rekurrente Netze (RNNs, LSTMs) Schritt für Schritt zu arbeiten, verarbeitet der Transformer Sequenzen vollständig parallel und stellt Abhängigkeiten durch einen Attention-Mechanismus dar, was insbesondere bei langen Kontexten Vorteile bringt.

3.2.1 Self-Attention

Im Zentrum des Transformers steht der *Self-Attention*-Mechanismus, der jedem Element einer Sequenz erlaubt, Informationen aus allen anderen Elementen zu beziehen. Für eine Eingabesequenz $\mathbf{X} \in \mathbb{R}^{T \times d}$ mit T Zeitschritten und Dimension d werden drei Projektionen berechnet:

$$\mathbf{Q} = \mathbf{XW}^Q \quad (\text{Queries}) \quad (3.1)$$

$$\mathbf{K} = \mathbf{XW}^K \quad (\text{Keys}) \quad (3.2)$$

$$\mathbf{V} = \mathbf{XW}^V \quad (\text{Values}) \quad (3.3)$$

Die Attention-Gewichte ergeben sich durch Skalierung des Skalarprodukts von Queries und Keys:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \mathbf{V} \quad (3.4)$$

Auf diese Weise kann jeder Frame einer Gebärdensequenz Informationen aus der gesamten Sequenz nutzen, was für das Erfassen globaler Bewegungsmuster entscheidend ist.

3.2.2 Positional Encoding

Da der Transformer keine eingebaute Reihenfolgeinformation besitzt, muss die zeitliche Position explizit kodiert werden. Dies erfolgt durch *Positional Encodings*, die zur Eingabe addiert werden. In dieser Arbeit werden *lernbare* Positions-Embeddings verwendet, die während des Trainings optimiert werden:

$$\mathbf{X}' = \mathbf{X} + \mathbf{E}_{\text{pos}} \quad (3.5)$$

wobei $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{T_{\text{max}} \times d}$ eine trainierbare Embedding-Matrix darstellt.

3.2.3 Vorteile gegenüber rekurrenten Architekturen

Transformer bieten mehrere Vorteile für die Gebärdenerkennung:

- **Parallelisierung:** Alle Positionen werden gleichzeitig verarbeitet, was das Training beschleunigt.
- **Lange Abhängigkeiten:** Direkte Verbindungen zwischen beliebigen Positionen vermeiden das Problem verschwindender Gradienten bei langen Sequenzen.
- **Interpretierbarkeit:** Attention-Gewichte lassen sich visualisieren und zeigen, welche Frames für die Klassifikation relevant sind.

3.3 Herausforderungen des Datensatzes

3.3.1 Long-Tail-Verteilung

Ein zentrales Problem bei der Gebärdenspracherkennung ist die ungleiche Verteilung der Trainingsbeispiele. Der verwendete Phoenix-Datensatz folgt einer *Power-Law*-Verteilung: Wenige sehr häufige Gebärden (z. B. REGION, MORGEN) dominieren, während die grosse Mehrheit der Klassen nur selten auftritt.

3.3.2 Gini-Koeffizient

Die Ungleichverteilung lässt sich mithilfe des Gini-Koeffizienten G quantifizieren. Für n Klassen mit Stichprobenanzahlen x_i gilt:

$$G = \frac{\sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|}{2n \sum_{i=1}^n x_i} \quad (3.6)$$

Ein hoher Gini-Wert zeigt eine starke Konzentration auf wenige „Head“-Klassen, während die meisten Klassen im „Tail“ unterrepräsentiert sind. In unserem Datensatz beträgt der Gini-Koeffizient $G = 0.81$. Solche Verteilungen erfordern spezielle Trainingsstrategien wie Focal Loss, Class-Balanced Loss oder Oversampling.

3.3.3 Visuelle Konfusion

Neben statistischer Imbalance erschwert die *phonologische Ähnlichkeit* die Klassifikation: Viele Gebärden unterscheiden sich lediglich in einem Parameter. So unterscheiden sich etwa die Gebärden für Himmelsrichtungen (NORD, SÜD, OST, WEST) oft nur in der Bewegungsrichtung bei ansonsten identischer Handform und -position.

Diese systematisch verwechslungsanfälligen Gruppen werden in dieser Arbeit als *Konfusionscluster* bezeichnet.

Dieses Kapitel beschreibt die technische Umsetzung und den iterativen Forschungsprozess des *SignNet*-Systems. Im Zentrum stehen die Begründungen der Designentscheidungen, die in mehreren Entwicklungsphasen getroffen wurden, um eine robuste und dynamische Gebärdenspracherkennung zu ermöglichen.

4.1 Datengrundlage und Klassenwahl

Die Auswahl geeigneter Daten war zentral für den Erfolg von *SignNet*. Da Gebärdensprache weit über das reine Buchstabieren hinausgeht, basiert das System auf zwei komplementären Säulen: dem statischen Fingeralphabet als kontrolliertem Einstiegspunkt und dem komplexen *PHOENIX*-Datensatz für die dynamische Gebärdenerkennung.

Als erster Schritt diente das deutsche Fingeralphabet (DGS). Es bot ein überschaubares, statisches Szenario, in dem sich die Präzision der Landmark-Extraktion systematisch beurteilen liess. Verwendet wurden Bilddaten zu 24 Handformen (A–Z, ohne J und Z, da diese Bewegungen beinhalten).

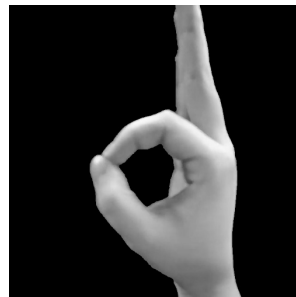


Abbildung 4.1: Beispielbild aus dem DGS-Fingeralphabet.

Die eigentliche Herausforderung bildete der *PHOENIX-2014*-Datensatz. Er umfasst Videoaufnahmen von Wettervorhersagen in Gebärdensprache und enthält ursprünglich über 1 200 Glossen. Die inhaltliche und visuelle Heterogenität des Materials — unterschiedliche Signer, Beleuchtungssituationen, Perspektiven und Tempi — macht den Datensatz realistisch, zugleich aber technisch anspruchsvoll. Für *SignNet* diente PHOENIX als massgebliche Grundlage, um unter praxisnahen Bedingungen Generalisierung, zeitliche Abhängigkeiten und Interaktionen zwischen Hand- und Gesichtsmerkmalen zu evaluieren. Aus diesen Eigenschaften ergaben sich konkrete Anforderungen an Augmentation, Feature-Engineering und balancierte Verlustfunktionen, die die weiteren Entwicklungsphasen prägten.

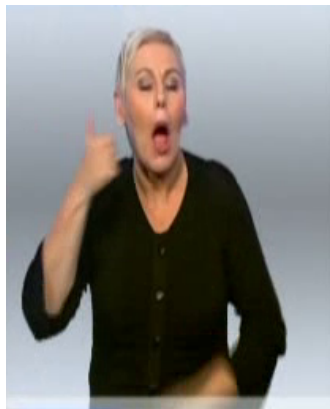


Abbildung 4.2: Beispiel-Frame aus PHOENIX-2014.

4.1.1 Filterkriterien und Stichprobenumfang

Aufgrund der starken Klassenimbalance (Long-Tail-Verteilung) wurde das Vokabular ab Phase 6 (siehe Abschnitt 4.2) gezielt eingeschränkt, um stabile Trainingsbedingungen zu schaffen:

- **Mindestanforderung:** Berücksichtigt wurden nur Klassen mit mindestens `MIN_SAMPLES_PER_CLASS` = 70 Beispielen.
- **Resultat:** Nach der Filterung blieben **146 Klassen** für die finale Klassifikation übrig.
- **Begründung:** So erhält das Modell pro Gebärde ausreichend Variabilität, um generalisierbare Merkmale zu lernen statt blosses Auswendiglernen.

4.1.2 Merkmalsextraktion mit MediaPipe

Um die Komplexität der Rohvideos zu reduzieren und Robustheit gegenüber Hintergrund und Beleuchtung zu erreichen, verwendeten wir die *MediaPipe*-Pipeline [10]. Statt direkt auf Pixelebene zu arbeiten, extrahiert diese Pipeline anatomisch relevante 3D-Landmarks für jedes Frame.

Die Extraktion erfolgt mittels dreier spezialisierter Submodelle, deren Ergebnisse zu einem kombinierten Merkmalsvektor zusammengeführt werden:

- **Face-Mesh:** 478 Landmarks für Mimik und Mundbild.
- **Hand-Tracking:** 21 Landmarks pro Hand (insgesamt 42 Punkte) für die Fingerstellung.
- **Pose-Modell:** 33 Landmarks zur Charakterisierung der Oberkörperhaltung.

Insgesamt ergeben sich 553 Punkte, je als Koordinatentripel (x, y, z) dargestellt, was einer Eingabedimension von 1659 Merkmalen pro Frame entspricht. Dieser flache Merkmalsvektor bildet die Grundlage für weiteres Feature Engineering wie Geschwindigkeiten und Bone-Vektoren. Die x - und y -Werte sind auf Bildbreite bzw. -höhe normiert $([0, 1])$, während z die relative Tiefeninformation zur Kamera darstellt.

4.2 Iterativer Forschungsansatz und Versuchsaufbau

Das System wurde schrittweise entwickelt; jede Phase baute auf den Erkenntnissen der Vorangegangenen auf, um identifizierte Schwachstellen gezielt zu adressieren.

4.2.1 Phase 1: Statische Baseline (CNN) – Fingeralphabet

Zunächst wurde die Erkennung des statischen Fingeralphabets (24 Klassen) untersucht:

- **Design:** Einsatz eines Convolutional Neural Networks (CNN) auf Bilddaten.
- **Ergebnis:** Testgenauigkeit von **99,6 %**.
- **Erkenntnis:** Hohe Genauigkeit, jedoch fehlte die Fähigkeit, zeitliche Dynamik der Gebärden zu erfassen.

4.2.2 Phase 2: Wechsel zum PHOENIX-Datensatz (Satzebene)

Mit Phase 2 begann die Arbeit an der eigentlichen „Königsdisziplin“: der Verarbeitung fließender Gebärdensprache auf Basis des **PHOENIX-2014**-Datensatzes. Dieser Datensatz bildete ab hier die Grundlage für alle weiteren Phasen.

- **Methode:** Transformer Architektur mit Connectionist Temporal Classification (CTC-Loss), um Alignment ohne explizite Segmentierung zu lernen.
- **Ergebnis:** Mit einer Word Error Rate (WER) von ca. **90 %** erwies sich der Ansatz als zu anspruchsvoll für die verfügbare Datenbasis.

4.2.3 Phase 3: Minimalistisches Satz-Modell (Debugging)

Um Konvergenzprobleme zu analysieren, entstand ein vereinfachtes Modell:

- **Methode:** MinimalCTCModel — schlanker Transformer-Encoder mit CTC-Loss.

- **Ergebnis:** Keine nennenswerte Lernfortschritte; folglich der Wechsel zur Klassifikation isolierter Wörter.

4.2.4 Phase 4: Übergang zu isolierten Wörtern (Top 10)

Der Fokus verschob sich auf einzelne Glossen, um Konvergenz sicherzustellen:

- **Methode:** LSTM-Architektur zur Verarbeitung von Landmark-Sequenzen; Vokabular: 10 häufigste Wörter.
- **Ergebnis:** Training-Accuracy **91,23 %**, Validierungs-Accuracy **81,15 %** — Bestätigung der Eignung der Landmarks für isolierte Worterkennung.

4.2.5 Phase 5: Skalierung auf Top 50 Wörter

Zur Prüfung der Skalierbarkeit wurde das Vokabular erweitert:

- **Methode:** LSTM mit Attention (`LSTM_with_Attention`) zur Fokussierung auf linguistisch relevante Frames.
- **Ergebnis:** Bei 50 Klassen sanken die Accuracies auf **56,84 %** (Training) bzw. **51,24 %** (Validierung).

4.2.6 Phase 6: Skalierung auf 146 Klassen und Wechsel zum Transformer

LSTM-Modelle zeigten bei wachsender Klassenanzahl Begrenzungen; daher stiegen wir auf Transformer um und erweiterten das Vokabular.

- **Konzept:** Ersetzen der LSTM-Struktur durch einen Transformer-Encoder. Self-Attention sollte zeitliche Abhängigkeiten der 1659-dimensionalen Landmark-Repräsentation besser erfassen.
- **Ergebnis:** Top-1-Validierungsgenauigkeit **53,94 %** — ein wichtiges Signal für erste Erfolge auf dem grossen Vokabular.

4.2.7 Phase 7: Ein Exkurs in skelettbasierte Graphen

Um die anatomische Struktur explizit zu modellieren, testeten wir skelettbasierte Graphen:

- **Experiment:** Abbildung anatomischer Verbindungen der Hand als Graph, mit der Hoffnung, physikalisch plausiblere Handformen zu erzwingen.
- **Ergebnis:** Validierungsgenauigkeit sank auf **31,56 %**. Rückblickend zeigte sich, dass starre Graphen die notwendige Variabilität der menschlichen Gebärdung zu stark einschränken.

4.2.8 Phase 8: Rückkehr zur optimierten Transformer-Architektur (v2)

Aus den Erkenntnissen von Phase 7 resultierte eine optimierte Transformer-Baseline:

- **Konzept:** Robustere Self-Attention-Baseline mit angepassten Hyperparametern.
- **Ergebnis:** Validierungsgenauigkeit stieg auf **66,61 %**.

4.2.9 Phase 9: Finales optimiertes Modell

Durch umfangreiches Hyperparameter-Tuning und verfeinertes Feature Engineering wurde die aktuelle Bestleistung erzielt:

- **Optimierung:** Label Smoothing, Augmentation und optimierte kinematische Features.
- **Resultat:** Finale Validierungsgenauigkeit **73,86 %** sowie Top-5-Accuracy **91,76 %**.

4.3 Vorverarbeitung und Datenreinigung

Robuste Vorverarbeitung war essenziell für stabile Modelle.

4.3.1 Umgang mit Fehlwerten und Verdeckungen (Occlusion)

MediaPipe ist anfällig für Verdeckungen (z. B. überlappende Hände). Massnahmen:

- **Numerische Stabilität:** Auftretende NaN-Werte werden durch 0.0 ersetzt, um Fehler in folgenden Berechnungen zu vermeiden.
- **Händigkeits-Invarianz:** Der `TransformerSignClassifierWithHandedness` berücksichtigt explizit die dominante Hand, um Händigkeit unabhängig zu machen.

4.4 Datenaugmentation

Zur Milderung des Long-Tail-Problems wurde eine mehrstufige Augmentationspipeline entwickelt, die sequenziell fünf Techniken anwendet.

4.4.1 Temporale Augmentation (TemporalAugmentation)

Mit 75 % Wahrscheinlichkeit angewendet; unterrepräsentierte Klassen werden stärker augmentiert:

- **Time Warping:** Streckung/Stauchung der Zeitachse (Faktor 0,8–1,25).
- **Temporal Dropout:** Entfernen einzelner Frames (Keep-Probability 0,85–0,98).
- **Global Scaling:** Skalierung der Sequenz (0,92–1,08).
- **Gaussian Noise:** Additives Rauschen ($\sigma = 0,008$) auf Landmark-Koordinaten.
- **Structured Channel Dropout:** Ausfall ganzer Feature-Blöcke (10 %).

4.4.2 Landmark-Occlusion-Augmentation

Mit 30 % Wahrscheinlichkeit werden realistische Okklusionsmuster simuliert:

- **Region Dropout (20 %):** Ausfall anatomischer Regionen (z. B. beide Hände vor Gesicht).
- **Temporal Dropout (15 %):** Tracking-Verlust über bis zu sieben Frames.
- **Sub-Region Dropout (25 %):** Ausfall feiner Merkmale (einzelne Finger, Augen, Mund).
- **Gradual Occlusion:** Fade-In/Fade-Out über 2–5 Frames mittels Alpha-Masking.

Okklusionen werden durch Nullsetzen der entsprechenden Koordinaten realisiert, nach anatomischer Plausibilität.

4.4.3 Mirror Augmentation

Horizontales Spiegeln (30 %):

- **Hand-Swapping:** Vertauschung der Landmark-Indizes beider Hände.
- **X-Koordinaten-Flip:** $x \rightarrow 1 - x$ für normierte Werte.
- **Pose-Pair-Swapping:** Vertauschung links/rechts relevanter Pose-Landmarks.

4.4.4 Weighted Sampling und Oversampling

Zur Reduktion von Class-Imbalance:

- **WeightedRandomSampler:** Die Sampling-Gewichte sind proportional zu $1/\sqrt{n_c}$, wobei n_c die Anzahl der Trainingssamples einer Klasse bezeichnet. Klassen mit wenigen Beispielen erhalten dadurch höhere Sampling-Wahrscheinlichkeiten, während häufig vertretene Klassen entsprechend seltener gezogen werden. Die Wurzel im Nenner sorgt für eine konservativere Gewichtung als $1/n_c$ und verhindert ein Überkompensieren selten vorkommender Klassen.
- **Explicit Oversampling:** Gezielte Duplikation seltener Klassen (z. B. „ZWEI“ 5×).

4.5 Feature Engineering: Kinematik und Geometrie

Die Klasse `EnhancedLandmarkFeatures` wandelt statische Landmark-Daten in einen dynamischen, hoch-dimensionalen Merkmalsraum zur Abbildung zeitlicher und geometrischer Variationen.

4.5.1 Kinematische Ableitungen

Dynamik wird über Geschwindigkeit $\vec{v}_t$ und optional Beschleunigung $\vec{a}_t$ erfasst (fps = 25):

$$\vec{v}_t = (\mathbf{p}_t - \mathbf{p}_{t-1}) \cdot \text{fps} \quad , \quad \vec{a}_t = (\vec{v}_t - \vec{v}_{t-1}) \cdot \text{fps} \quad (4.1)$$

Geschwindigkeitsvektoren werden per Z-Score-Normalisierung ($\mu = 0$, $\sigma = 1$) entlang der Sequenz standardisiert.

4.5.2 Skelett-Geometrie und Bone-Vektoren

Bone-Vektoren zwischen anatomisch verbundenen Gelenken nach MediaPipe-Hand-Topologie:

$$\vec{b} = \mathbf{p}_{\text{joint_A}} - \mathbf{p}_{\text{joint_B}} \quad (4.2)$$

Diese Vektoren sind translationsinvariant. Während des Trainings werden Bone-Längen mit 30 % Wahrscheinlichkeit zufällig skaliert (Faktor 0,9–1,1), um anatomische Unterschiede zu simulieren.

4.5.3 Distanzmerkmale und Normalisierung

Ergänzend werden euklidische Abstände d zwischen kritischen Landmark-Paaren berechnet:

$$d = \|\mathbf{p}_{\text{joint_A}} - \mathbf{p}_{\text{joint_B}}\|_2 \quad (4.3)$$

Zur Positionsinvarianz werden alle Koordinaten vor Merkmalsextraktion relativ zum Handgelenk (wrist, Landmark 0) verschoben.

4.6 Modellarchitektur: Transformer-Encoder

Der Transformer wurde gewählt, weil er globale Abhängigkeiten effizient modelliert:

- **Self-Attention:** Ermöglicht dem Modell, irrelevante Phasen auszublenden und sich auf linguistisch relevante Bewegungen zu konzentrieren.
- **Padding-Masking:** Mittels `PadCollate` und `seq_lengths` werden variable Videolängen korrekt behandelt, ohne dass Padding die Berechnung verfälscht.

4.6.1 Spezifikation der Transformer-Architektur

Um Overfitting vorzubeugen, wurde eine kompakte Konfiguration gewählt. Die reduzierte Modellgröße fördert die Generalisierung auf dem Validierungsset. Zur Optimierung des Trainingsprozesses wurde ein *SequentialLR*-Scheduler mit zwei Phasen implementiert. In der ersten Phase sorgt ein *LinearLR*-Warmup über 30 Epochen für einen kontrollierten Anstieg der Lernrate von 10 % des Basiswerts (10^{-4}) bis zum Maximum. Diese Massnahme reduziert die bei Transformern häufig auftretende Gradienteninstabilität in der Frühphase.

Anschließend übernimmt ein *CosineAnnealingWarmRestarts*-Scheduler, der die Lernrate zyklisch bis auf ein Minimum von 10^{-7} absenkt. Die Parameter $T_0 = 100$ und $T_{\text{mult}} = 2$ bewirken, dass sich die Periodendauer nach jedem Restart verdoppelt. Dadurch kann das Modell tiefer in globale Minima der Verlustfunktion einrasten, während die gelegentlichen Erhöhungen der Lernrate helfen, lokale Optima zu überwinden.

Tabelle 4.1: Hyperparameter der Architektur und des Trainingsprozesses (Phase 9)

Parameter	Wert	Begründung
<code>input_size</code>	1659	Resultiert aus 553 Landmarks mit je 3 Koordinaten (x, y, z) .
<code>hidden_size</code>	256	Reduktion zur Vermeidung von Overfitting bei begrenzter Datenmenge.
<code>num_layers</code>	4	Balance zwischen Tiefe und Stabilität.
<code>num_heads</code>	8	Ermöglicht mehrere parallele Attention-Heads.
<code>dropout_rate</code>	0,65	Starke Regularisierung zur Reduktion der Differenz zwischen Trainings- und Validierungsleistung.
<code>weight_decay</code>	10^{-2}	L2-Regularisierung zur Verbesserung der Generalisierungsfähigkeit.
Optimierer	AdamW	Robustheit gegenüber spärlichen Gradienten und entkoppeltes Weight Decay.
Lernrate (Basis)	10^{-4}	Stabiler Ausgangspunkt für den Lernprozess.
Scheduler	SequentialLR	Kombination aus linearem Warmup (30 Epochen) und <i>CosineAnnealingWarmRestarts</i> ($T_0 = 100$).
Label Smoothing	0,05	Verhindert eine zu hohe Konfidenz des Modells und verbessert die Kalibrierung.

In diesem Kapitel werden die quantitativen Ergebnisse der neun Entwicklungsphasen präsentiert. Die Zahlen beruhen auf MLflow-Tracking und abschliessenden Evaluationen der jeweiligen Modellvarianten.

5.1 Vergleich der Entwicklungsphasen

Die Performance der unterschiedlichen Architekturen wurde während des gesamten Projektverlaufs systematisch dokumentiert. Tabelle 5.1 fasst die Validierungsgenauigkeiten von der statischen Vorphase bis zum finalen Transformer-Modell zusammen.

Tabelle 5.1: Übersicht der erreichten Genauigkeiten pro Phase

Phase	Architektur	Fokus	Validierungsgüte
1	CNN	Statisches Alphabet (24 Klassen)	99,60 %
2	Transformer + CTC	Satzebene (CTC-Loss)	90,00 % (WER)
3	MinimalCTC	Debugging Satzebene	keine Konv.
4	LSTM	Top 10 Wörter	81,15 %
5	LSTM + Attention	Top 50 Wörter	51,24 %
6	Transformer v1	Dynamische Gebärden (146 Klassen)	53,94 %
7	Skeleton-Graph	Anatomische Graphen-Pipeline	31,56 %
8	Transformer v2	Self-Attention Baseline	66,61 %
9	Transformer Final	Optimiertes Feature Engineering	73,86 %

5.2 Datensatz-Statistik und Split-Verhältnis

Für Phase 9 wurde der gefilterte Datensatz (146 Klassen, $N \geq 70$) stratifiziert in Training und Validierung im Verhältnis 80:20 aufgeteilt. Auf die Ausgliederung eines separaten, gänzlich unberührten Test-Datensatzes wurde dabei zugunsten einer breiteren Validierungsbasis verzichtet.

Tabelle 5.2: Verteilung der Stichproben (80/20-Split)

Datensatz-Teil	Anzahl Samples	Anteil [%]
Trainingsset	22.628	80,0 %
Validierungsset	5.657	20,0 %
Gesamtanzahl (gefiltert)	28.285	100,0 %

5.3 Analyse des Trainingsverlaufs

Der Verlauf von Loss und Accuracy des finalen Modells (Phase 9) wurde über die gesamte Trainingsdauer aufgezeichnet. In Abbildung 5.1a ist zu sehen, dass sich Trainings- und Validierungs-Loss konsistent nach unten bewegen. Die Accuracy-Kurven (Abbildung 5.1b) zeigen ein stetiges Ansteigen mit einem anschliessenden Plateau. Im stabilen Bereich beträgt die Differenz zwischen Trainings- und Validierungswerten etwa 11,55 Prozentpunkte.

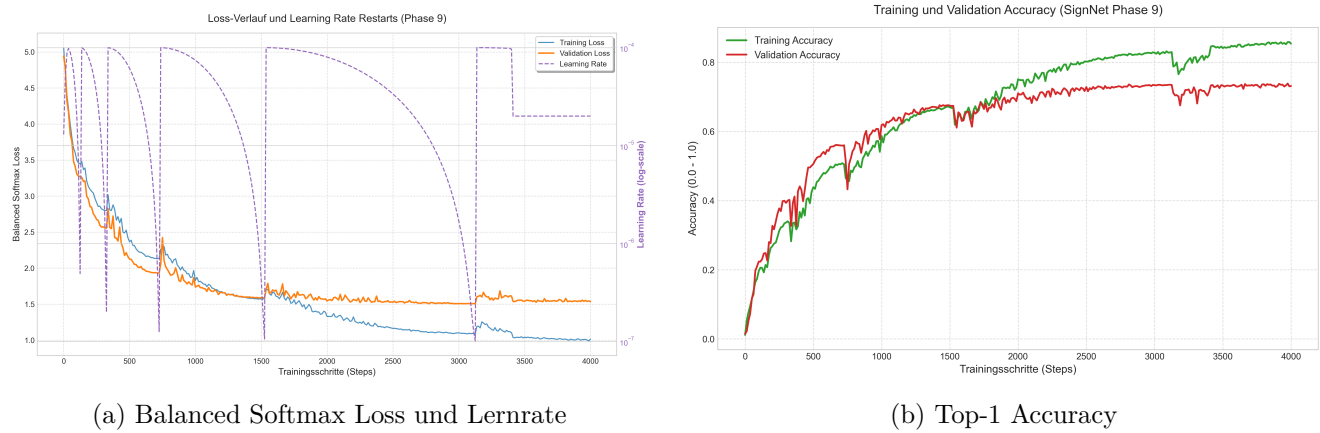


Abbildung 5.1: Trainingsmetriken der Phase 9: (a) zeigt den Verlauf des *Balanced Softmax Loss* im Verhältnis zur zyklischen Lernrate (lila gestrichelt). Die periodischen Restarts des *CosineAnnealing*-Schedulers ($T_0 = 100$, $T_{\text{mult}} = 2$) unterstützen das Verlassen lokaler Optima und fördern eine tiefere Konvergenz. (b) verdeutlicht die Stabilisierung der Validierungsgenauigkeit bei 73,86 %.

5.4 Quantitative Ergebnisse

Die Leistung in Phase 9 wurde mit Top-1- und Top-5-Accuracy auf dem Validierungsset bewertet. Für das Vokabular von 146 Klassen ergeben sich die in Tabelle 5.3 angegebenen Kennzahlen.

Tabelle 5.3: Globale Klassifikationsergebnisse (Phase 9)

Metrik	Wert
Training Accuracy	85,41 %
Validierungs-Accuracy (Top-1)	73,86 %
Top-5 Accuracy	91,76 %
Anzahl Validierungs-Samples	5.657
Anzahl Klassen ($N \geq 70$)	146
Gini-Index (nach Filterung)	0,32

5.5 Konfidenz-Analyse und Kalibrierung

Die Auswertung der Softmax-Konfidenzen (Tabelle 5.4) zeigt klare Unterschiede zwischen korrekten und fehlerhaften Vorhersagen: Korrekte Vorhersagen sind im Mittel deutlich sicherer.

Tabelle 5.4: Statistische Auswertung der Modell-Konfidenz

Kategorie	Durchschnittliche Konfidenz
Korrekte Vorhersagen	87,37 %
Fehlklassifikationen	57,19 %
Anteil Fehler mit hoher Konfidenz ($> 0,8$)	5,30 %

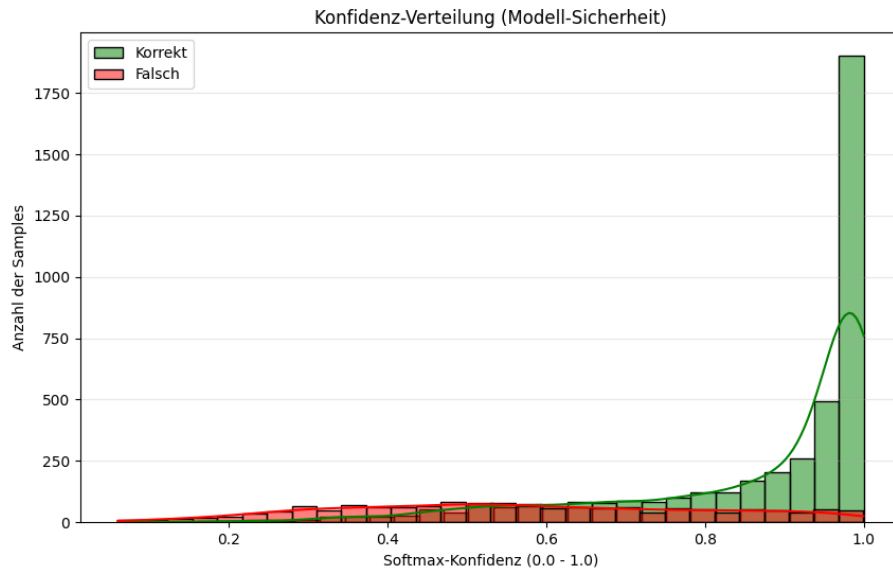


Abbildung 5.2: Konfidenz-Verteilung für korrekte (grün) und falsche (rot) Vorhersagen.

5.6 Detaillierte Fehleranalyse

Die Konfusionsmatrix der zehn Klassen mit dem niedrigsten F1-Score (Abbildung 5.3) macht wiederkehrende Verwechslungsmuster sichtbar. Diese Häufungen deuten auf systematische linguistische und visuelle Überschneidungen zwischen bestimmten Klassen hin, die gezielte Nachbearbeitung erfordern.

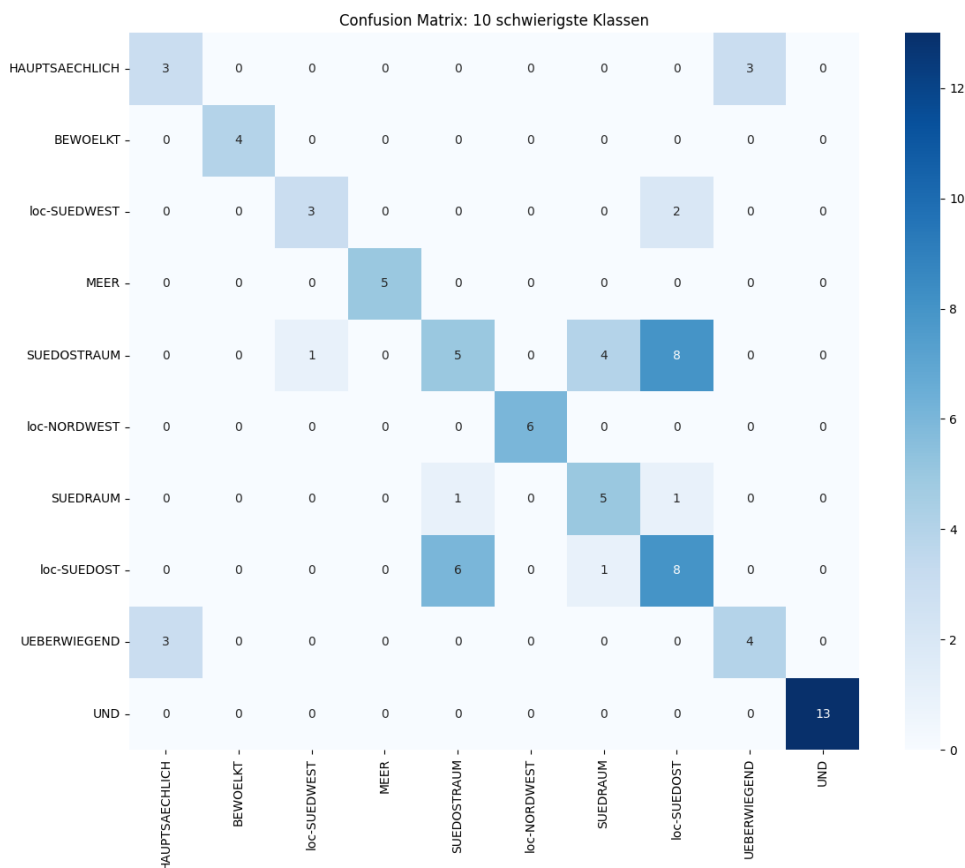


Abbildung 5.3: Konfusionsmatrix der zehn Klassen mit der geringsten Trennschärfe.

5.6.1 Evaluation nach Klassenhäufigkeit (Buckets)

Zur Überprüfung der Robustheit gegenüber der Long-Tail-Verteilung wurde eine Bucket-Analyse durchgeführt (siehe Abbildung 5.4). Die Ergebnisse zeigen, dass selbst seltene Klassen ($N < 100$) eine Top-1-Accuracy von 65,8 % erreichen, während häufige Klassen ($N > 300$) bei 80,3 % liegen.

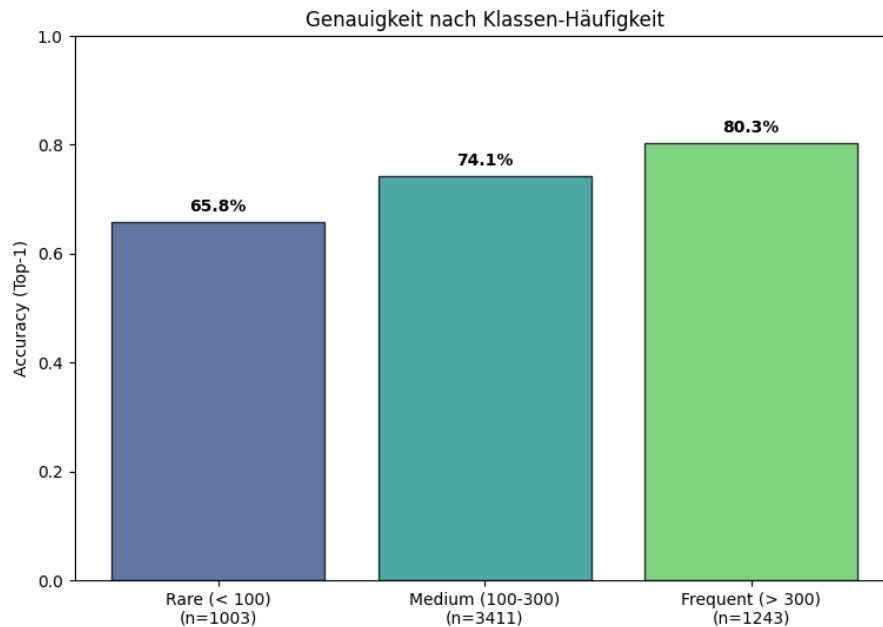


Abbildung 5.4: Genauigkeit gruppiert nach der Häufigkeit der Klassen im Trainingsset.

Diskussion

In diesem Kapitel werden die Ergebnisse der finalen Phase 9 im Zusammenhang mit der Forschungsfrage und den theoretischen Grundlagen interpretiert. Ziel ist es, die in Kapitel 5 dargestellten Metriken mit den linguistischen und architektonischen Herausforderungen in Beziehung zu setzen.

6.1 Interpretation der Klassifikationsleistung

Die finale Evaluation ergab eine Top-1-Accuracy von 73,86 % für das Vokabular von 146 Klassen. Im Vergleich zur Phase-6-Baseline (53,94 %) verdeutlicht dies den Nutzen des verfeinerten Feature-Engineerings und der finalen Transformer-Architektur. Besonders aussagekräftig ist die Top-5-Accuracy von 91,76 %. Die Differenz von knapp 17 Prozentpunkten bedeutet, dass die richtige Gebärde in den meisten Fällen in einem engen Kandidatenkreis enthalten ist.

6.1.1 Robustheit gegenüber Klassenungleichgewicht (Bucket-Analyse)

Ein zentraler Nachweis für die Qualität des Modells ist die Analyse der Genauigkeit in Abhängigkeit von der Trainingsstichprobe (siehe Abbildung 5.4 in Kapitel 5). Die Ergebnisse verdeutlichen, dass das System nicht lediglich die häufigsten Gebärden „auswendig lernt“, sondern eine bemerkenswerte Generalisierungsleistung über das gesamte Spektrum erbringt:

- **Seltene Klassen (Rare, < 100 Samples):** Trotz der geringen Datenbasis erreicht das Modell hier eine Accuracy von 65,8 %. Dieser Wert ist ein direkter Erfolg des *Balanced Softmax Loss*.
- **Häufige Klassen (Frequent, > 300 Samples):** Mit einer Genauigkeit von 80,3 % zeigt sich, dass das Modell das volle Potenzial der verfügbaren Variabilität ausschöpft.

Diese Verteilung bestätigt, dass die Kombination aus physikalisch motiviertem Feature-Engineering und einer balancierten Verlustfunktion die strukturellen Defizite des PHOENIX-Datensatzes (Gini-Index 0,81 vor Filterung) effektiv kompensiert.

6.2 Evaluation der Modellstabilität und Generalisierungsfähigkeit

Ein zentraler Analysepunkt ist das Verhalten der Lernkurven in Hinblick auf Generalisierung und Optimierungspotential.

6.2.1 Detaillierte Loss-Analyse

Der Verlauf des *Balanced Softmax Loss* liefert Hinweise auf die Trainingsstabilität. Wie in Abbildung 5.1a sichtbar, sinken Training- und Validation-Loss über die Trainingsdauer gleichmässig.

- **Konvergenz:** Beide Kurven erreichen gegen Ende ein Plateau — der Training-Loss bei etwa 1,1, der Validation-Loss bei rund 1,55.
- **Kein typisches Overfitting-Signal:** Ein wieder ansteigender Validation-Loss bei weiter fallendem Training-Loss wäre ein klares Overfitting-Indiz. Da die Kurven hier weitgehend parallel verlaufen, spricht das gegen eine destruktive Überanpassung.

6.2.2 Interpretation des Accuracy-Gaps und Early Stopping

Der Unterschied zwischen finaler Training-Accuracy (85,41 %) und Validation-Accuracy (73,86 %) beträgt 11,55 Prozentpunkte. Zeitliche Detailbetrachtungen zeigen Optimierungsmöglichkeiten:

- **Frühes Plateau:** Die Validierungsgenauigkeit erreichte bereits um Schritt 2700 etwa 73 % und verbesserte sich danach nur noch geringfügig.

- **Early Stopping als Hebel:** Würde das Training bei Schritt 2700 gestoppt, liesse sich der Generalization-Gap auf etwa 9 % reduzieren und Rechenressourcen sparen.
- **Rolle der Regularisierung:** Dass der Gap trotz weiterem Training nicht dramatisch anstieg, ist ein Verdienst der starken Regularisierung (Dropout 0,65).

6.2.3 Fazit zur Generalisierung

Insgesamt ist die Generalisierung robust zu bewerten: Die parallelen Loss-Verläufe deuten auf stabile, aussagekräftige Repräsentationen hin. Strengere Abbruchkriterien (Early Stopping) könnten in zukünftigen Durchläufen sowohl Effizienz als auch Generalisierung weiter verbessern.

6.3 Analyse linguistischer Konfusionscluster

Die Konfusionsmatrix in Abbildung 5.3 zeigt, dass Fehlklassifikationen systematischen Mustern folgen, statt rein zufällig zu sein:

- **Morphologische Varianten und Classifier:** Verwechslungen treten häufig zwischen Stammgebärden und ihren klassifikatorischen (**cl-**) oder lokalisatorischen (**loc-**) Erweiterungen auf.
- **Visuelle Minimalpaare:** Paare wie **ABEND** und **NACHT** teilen oft Handform und Ausführungsort; die Unterscheidung beruht auf non-manualen Markern (Mimik), die aktuell offenbar noch zu wenig gewichtet werden.
- **Richtungsambiguitäten:** Verwechslungen zwischen **loc-NORD** und **NORDRAUM** verdeutlichen die Herausforderung, Richtungsinformation konsistent zu kodieren.

Diese Muster geben konkrete Hinweise, wo gezielte Feature-Gewichtung oder zusätzliche Modalitäten besonders wirksam sein könnten.

6.4 Modellstabilität und Konfidenz-Evaluation

Die Konfidenzverteilung in Abbildung 5.2 spricht für eine sinnvolle Kalibrierung des Modells:

- **Trennschärfe:** Korrekte Vorhersagen liegen im Mittel bei 87,37 % Konfidenz, falsche Vorhersagen bei 57,19 % — ein klares Signal für verlässliche Unsicherheitsabschätzungen.
- **Sichere Fehlvorhersagen:** Der geringe Anteil von 5,30 % „sicheren Fehlern“ (Konfidenz > 0,8 bei falscher Vorhersage) zeigt die Robustheit der Architektur.

6.5 Methodische Reflexion: Das Validierungs-Dilemma

In Phase 9 entschieden wir uns für einen binären Split (Train/Validation) statt der klassischen Dreiteilung (Train/Validation/Test). Diese Wahl erhöhte die Stabilität der Metriken bei stark unbalancierten Klassen, bringt jedoch methodische Risiken mit sich.

6.5.1 Statistische Relevanz vs. Unabhängigkeit

Ein isoliertes Test-Set ist Standard, um Generalisierung auf unbekannte Daten zu belegen. Im Long-Tail-Bereich mit nur 70 Stichproben pro Klasse hätte eine 70:15:15-Aufteilung jedoch lediglich etwa zehn Testbeispiele ergeben. Eine einzige Fehlklassifikation entspräche dort bereits ~ 10 % Genauigkeitseinbruch und würde die Aussagekraft verzerren. Der 80:20-Split stellt dagegen mindestens 14 Validierungsbeispiele pro rare Klasse bereit und stabilisiert die Metriken.

6.5.2 Risiko des Hyperparameter-Overfittings

Die iterative Abstimmung von Architektur und Hyperparametern anhand der Validierung birgt *Selection Bias* und potenzielles Hyperparameter-Overfitting. Zukünftige Arbeiten sollten daher die erreichte Genauigkeit von 73,86 % auf einem vollständig externen Datensatz (andere Signer, veränderte Bedingungen)

verifizieren. Im Rahmen dieser Arbeit wogen jedoch geringere Varianz und belastbarere Metriken in 146 Klassen schwerer als ein kleiner, potenziell irreführender Testsatz.

6.6 Architektonische Einflüsse und Limitationen

Der Erfolg in Phase 9 lässt sich wesentlich auf die Rückkehr zur Transformer-Architektur mit kompakter `hidden_size=256` zurückführen. Während starre Skeleton-Graphen zu wenig Flexibilität geboten haben, ermöglicht Self-Attention eine adaptive Gewichtung zeitlicher Abhängigkeiten.

Gleichzeitig bleibt die phonologische Nähe vieler Gebärden die zentrale Limitation. Ein praktikabler nächster Schritt wäre, Mundbild-Features (Face-Mesh) innerhalb der Cross-Attention stärker zu gewichten, um verbleibende Minimalpaar-Konfusionen gezielt zu reduzieren.

Fazit und Ausblick

7.1 Zusammenfassung der Ergebnisse

Ziel dieser Arbeit war es, zu demonstrieren, dass zuverlässige Gebärdenspracherkennung auch unter schwierigen Bedingungen möglich ist — etwa bei stark ungleich verteilten Daten oder bei visuell ähnlichen Gebärden. Das *SignNet*-Framework hat sich dabei als zentrale Komponente erwiesen: Die Kombination eines Transformer-Modells mit physikalisch motiviertem Feature-Engineering führte zu klaren Verbesserungen der Erkennungsleistung.

Während die erste Transformer-Phase (Phase 6) noch eine Validierungsgenauigkeit von 53,94 % zeigte, stieg diese mit der Integration kinematischer und geometrischer Merkmale in Phase 9 auf **73,86 %**. Ebenfalls bemerkenswert ist die Top-5-Genauigkeit von **91,76 %** — in über neun von zehn Fällen liegt die korrekte Gebärde unter den fünf wahrscheinlichsten Vorhersagen. Diese Resultate belegen, dass die *EnhancedLandmarkFeatures* einen entscheidenden Beitrag zur besseren Abbildung dynamischer Gebärdeneigenschaften geleistet haben.

7.2 Technische Würdigung

Die massgeblichen technischen Herausforderungen lagen in der Verarbeitung der 1659-dimensionalen Landmark-Daten und im Umgang mit der starken Klassenungleichheit des PHOENIX-Datensatzes.

- **Feature Engineering:** Die Klasse `EnhancedLandmarkFeatures` machte es möglich, Bewegungscharakteristika wie Geschwindigkeit, Beschleunigung und Bone-Vektoren explizit zu modellieren — also Informationen, die über die statische Pose hinausgehen.
- **Generalisierung und Regularisierung:** Eine kompakte Transformer-Konfiguration zusammen mit starkem Dropout sorgte für stabile Trainingsverläufe. Der verbleibende Abstand von rund 12 % zwischen Trainings- und Validierungsleistung ist angesichts der hohen intra-klassigen Variabilität vertretbar.
- **Umgang mit Imbalance:** Die Beschränkung des Vokabulars auf 146 Klassen (jeweils ≥ 70 Beispiele) reduzierte die extreme Schiefe und schaffte eine belastbare Basis für das Modelltraining.

7.3 Ausblick

Trotz erzielter Fortschritte bleiben mehrere Anschlussfragen offen. Besonders vielversprechend ist die stärkere Fusion verschiedener Modalitäten: Die Verknüpfung von Skelettdaten mit optischem Fluss aus den RGB-Videos könnte Bewegungsdynamiken erfassen, die allein anhand von Landmarks schwer zu erfassen sind.

Unsere Fehleranalyse zeigte zudem, dass das reine Skelettmodell bei optischen Minimalpaaren wie **ABEND** und **NACHT** an Grenzen stösst. In solchen Fällen sind non-manuelle Marker (Mimik, Mundbild) entscheidend. Eine stärkere Gewichtung des Face-Mesh in der Architektur dürfte die Unterscheidung verbessern. Architektonisch könnte ein lernbares Gating in einem hierarchischen System helfen, zwischen Stammgebärden und komplexen Classifier-Varianten zielgerichteter zu entscheiden. Langfristiges Ziel bleibt die Überführung der isolierten Worterkennung in eine kontinuierliche Gebärdensprachübersetzung (Sign Language Translation). Ein ambitioniertes, aber realistisches Folgeprojekt für *SignNet*.

7.4 Persönliches Fazit Andrei

Die Arbeit an dieser Thesis war geprägt von der Arbeit mit realen, unvollständigen Daten und bot viele Lernmöglichkeiten. Im Folgenden beschreibe ich, was gut lief, was ich anders machen würde und was ich gelernt habe.

7.4.1 Was habe ich gut gemacht?

Die Projektarbeit hat mir gezeigt, dass systematisches Vorgehen der Schlüssel zum Erfolg ist. Besonders stolz bin ich auf die schrittweise Entwicklung des Projekts: Der Übergang vom statischen Fingeralphabet (99,6 % Accuracy auf 24 Gesten) zur kontinuierlichen Gebärdenerkennung auf dem RWTH-PHOENIX-2014 Datensatz war technisch anspruchsvoll, aber durch das schrittweise Vorgehen machbar.

Die **Datenanalyse vor architektonischen Entscheidungen** hat sich als richtig erwiesen. Die systematische Untersuchung der MediaPipe-Landmarks zeigte beispielsweise, dass die Z-Koordinaten zu 80–98 % nahezu Nullwerte enthielten – diese Erkenntnis ermöglichte eine Reduktion der Feature-Anzahl von 1'659 auf 1'086 Features ohne Genauigkeitsverlust.

Ein weiterer Erfolg war der **Umgang mit der extremen Klassenimbalance** (593:1 Verhältnis zwischen häufigster und seltenster Klasse). Die Implementierung von Balanced Softmax Loss in Kombination mit gezieltem Oversampling für seltene Klassen führte zu einer Gesamtverbesserung von 12,1 % (61,75 % auf 73,86 % Accuracy). Besonders beeindruckend war die 18,2 %-Verbesserung bei Klassen mit weniger als 100 Samples.

Die Entwicklung eines **Bucket-basierten Evaluationssystems** – inspiriert durch Feedback von Martin – ermöglichte eine genaue Analyse der Modelleistung über verschiedene Stichprobengruppen hinweg. Dies half, gezielt Schwachstellen zu erkennen und zu beheben.

Schliesslich war die **konsequente Nutzung von MLflow** für Experiment-Tracking wertvoll. Die Dokumentation aller Hyperparameter, Metriken und Modell-Artefakte ermöglichte nicht nur Reproduzierbarkeit, sondern auch fundierte Entscheidungen für Folge-Experimente.

7.4.2 Was würde ich anders machen?

Die grösste Lektion kam durch einen schmerzhaften **Train/Inference-Mismatch**: Der Landmark-Extraktor für das Training nutzte separate MediaPipe-Modelle (Hands, Face, Pose) mit expliziter Handedness-Erkennung (LEFT immer auf Index 0–62, RIGHT auf 63–125), während die GUI-Demo MediaPipe Holistic mit einfacher Nummerierung verwendete. Dieser grundlegende Unterschied führte dazu, dass das Modell im Demo-Modus praktisch blind war – teilweise nur 14 von 94 Frames korrekt verarbeitete. Rückblickend hätte ich von Anfang an auf **konsistente Pipelines** zwischen Training und Inference achten müssen. Die Handedness-Erkennung hätte schon früh eingebaut werden sollen, nicht erst nach dem Erkennen des Problems.

Auch beim Modell-Training gab es Lernmomente: Ein erster Optimierungsversuch mit erhöhter Regularisierung (Dropout 0,5, niedrige Learning Rate $5 \cdot 10^{-5}$) führte zu **Underfitting** statt der erhofften besseren Generalisierung – der Validation Loss stieg von 2,89 auf 4,88. Die Balance zwischen Regularisierung und Modellkapazität erfordert sorgfältige Abstimmung.

Im Rückblick würde ich der **anfänglichen Datenbereinigung** und der Korrektur von Gloss-Label-Inkonsistenzen mehr Zeit widmen, da diese die Basis für jede erfolgreiche Klassifizierung bilden. Eine zentrale Erkenntnis war, dass die Qualität der Vorverarbeitung (Landmark-Extraktion und Ausrichtung) oft einen grösseren Einfluss auf die finale Accuracy hat als die reine Komplexität der Modellarchitektur. Bei der **Dokumentation** hätte ich früher und regelmässiger arbeiten sollen. Die Thesis war zwar zu 70–80 % strukturiert, enthielt aber inkonsistente Accuracy-Zahlen zwischen verschiedenen Abschnitten – ein Zeichen dafür, dass paralleles Schreiben und Experimentieren ohne enge Abstimmung zu Fehlern führt.

7.4.3 Was habe ich gelernt?

Technisch Die Arbeit mit dem RWTH-PHOENIX-Datensatz gab mir ein gutes Verständnis für die Herausforderungen bei realen, unbalancierten Datensätzen. *Balanced Softmax Loss* erwies sich als deutlich effektiver als einfache Oversampling-Strategien allein – die Kombination beider Techniken war der Schlüssel zum Erfolg.

MediaPipe ist ein mächtiges Tool, aber seine Feinheiten – insbesondere die unterschiedliche Handhabung von Handedness zwischen Holistic- und separaten Hand-Modellen – muss man genau verstehen.

Die Erkenntnis, dass LEFT-Hände konsistent an definierten Indizes gespeichert werden müssen, war entscheidend für reproduzierbare Ergebnisse.

Die Transformer-Architektur mit 256 Hidden Units, 4 Layern und 8 Attention Heads (~4M Parameter) erwies sich als guter Kompromiss zwischen Performance und Overfitting-Risiko. Das ursprüngliche, grössere Modell (512h/6L/8heads, ~21M Parameter) zeigte einen 16 %-Train/Validation-Gap.

Besonders lehrreich war auch die Implementierung des *MisclassificationAnalyzer*, der mir half, die inhaltliche und visuelle Ähnlichkeit von Gebärden besser zu verstehen.

Prozessbezogen Der wissenschaftliche Arbeitsprozess – Hypothese aufstellen, Experiment durchführen, Ergebnisse analysieren, Schlüsse ziehen – ist keine theoretische Methode, sondern praktische Notwendigkeit. Die „gescheiterten“ Experimente (V1 mit Overfitting, V2 mit Underfitting) waren keine Rückschläge, sondern lieferten wertvolle Erkenntnisse für die finale V3-Konfiguration.

Cross-Platform-Entwicklung zwischen Windows (Entwicklung) und Linux (vast.ai Training) erfordert sorgfältige Trennung von Pfaden und Konfiguration. Die Verwendung von `pathlib` und umgebungsabhängigen Config-Klassen sparte später viel Debugging-Zeit.

Persönlich Das Projekt hat mir gezeigt, dass Machine Learning zu 80 % Data Engineering und Pipeline-Konsistenz ist, und nur zu 20 % Modell-Architektur. Diese Lektion werde ich in jedem zukünftigen ML-Projekt anwenden.

7.5 Persönliches Fazit Roman

7.5.1 Was habe ich gut gemacht?

Ich bin stolz auf das Feature Engineering: Die explizite Berechnung physikalischer Grössen wie Geschwindigkeit und Beschleunigung war ein Wendepunkt und trug massgeblich zur Erreichung von über 73 % Genauigkeit bei. MLflow-Monitoring half, Experimentverläufe nachzuvollziehen.

7.5.2 Was würde ich anders machen?

Frühere, tiefere Fehleranalyse und kreativere Datenaugmentationen (mehr Variation in Bewegung und Perspektive) würden helfen, häufige Verwechslungen schneller zu beheben und das Modell robuster zu machen.

7.5.3 Was habe ich gelernt?

Ein Transformer ist kein Allheilmittel ohne solide Datenbasis. Datenqualität und Vorverarbeitung sind oft entscheidender als die reine Modellkomplexität. Schlechte Experimente sind wertvolle Hinweise für die nächsten sinnvollen Schritte.

Literaturverzeichnis

- [1] Richard Bowden, David Windridge, Timor Kadir, Andrew Zisserman, and Michael Brady. A linguistic feature vector for the visual interpretation of sign language. In *European Conference on Computer Vision (ECCV)*, pages 390–401, 2004. 3
- [2] Necati Cihan Camgöz, Simon Hadfield, Oscar Koller, Hermann Ney, and Richard Bowden. Neural sign language translation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7784–7793, 2018. 4
- [3] Zhe Cao, Gines Hidalgo, Tomas Simon, Shih-En Wei, and Yaser Sheikh. Openpose: Realtime multi-person 2d pose estimation using part affinity fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(1):172–186, 2019. 4
- [4] João Carreira and Andrew Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6299–6308, 2017. 3
- [5] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9268–9277, 2019. 4
- [6] Konstantinos M. Dafnis, Evgenia Chroni, Carol Neidle, and Dimitris N. Metaxas. Bidirectional skeleton-based isolated sign recognition using graph convolutional networks. In *Proceedings of the 13th Conference on Language Resources and Evaluation (LREC 2022)*, pages 7328–7338, Marseille, 2022. European Language Resources Association (ELRA). 3
- [7] Oscar Koller, Jens Forster, and Hermann Ney. Continuous sign language recognition: Towards large vocabulary statistical recognition systems handling multiple signers. *Computer Vision and Image Understanding*, 141:108–125, December 2015. i, 4
- [8] Dongxu Li, Cristian Rodriguez, Xin Yu, and Xiaopeng Hong. Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1459–1469, 2020. 4
- [9] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017. 4
- [10] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hber, Fan Zhang, Zhuo Wu, Andrey Vakunov, and Matthias Grundmann. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019. 4, 10
- [11] Lionel Pigou, Sander Dieleman, Pieter-Jan Kindermans, and Benjamin Schrauwen. Sign language recognition using convolutional neural networks. In *European Conference on Computer Vision (ECCV) Workshops*, pages 572–578, 2014. 3
- [12] Jiawei Ren, Cunjun Yu, Xiao Ma, Haiyu Zhao, and Shuai Yi. Balanced meta-softmax for long-tailed visual recognition. In *Advances in Neural Information Processing Systems*, volume 33, pages 4175–4186, 2020. 5
- [13] Thad Starner, Joshua Weaver, and Alex Pentland. Real-time american sign language recognition using desk and wearable computer based video. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 20, pages 1371–1375, 1998. 3
- [14] William C. Stokoe. *Sign Language Structure: An Outline of the Visual Communication Systems of the American Deaf*. Number 8 in Studies in Linguistics: Occasional Papers. University of Buffalo, 1960. 6

- [15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017. 3, 7
- [16] Christian Vogler and Dimitris Metaxas. A framework for recognizing the simultaneous aspects of american sign language. In *Computer Vision and Image Understanding*, volume 81, pages 358–384, 2001. 3
- [17] Sijie Yan, Yuanjun Xiong, and Dahua Lin. Spatial temporal graph convolutional networks for skeleton-based action recognition. In *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018. 4